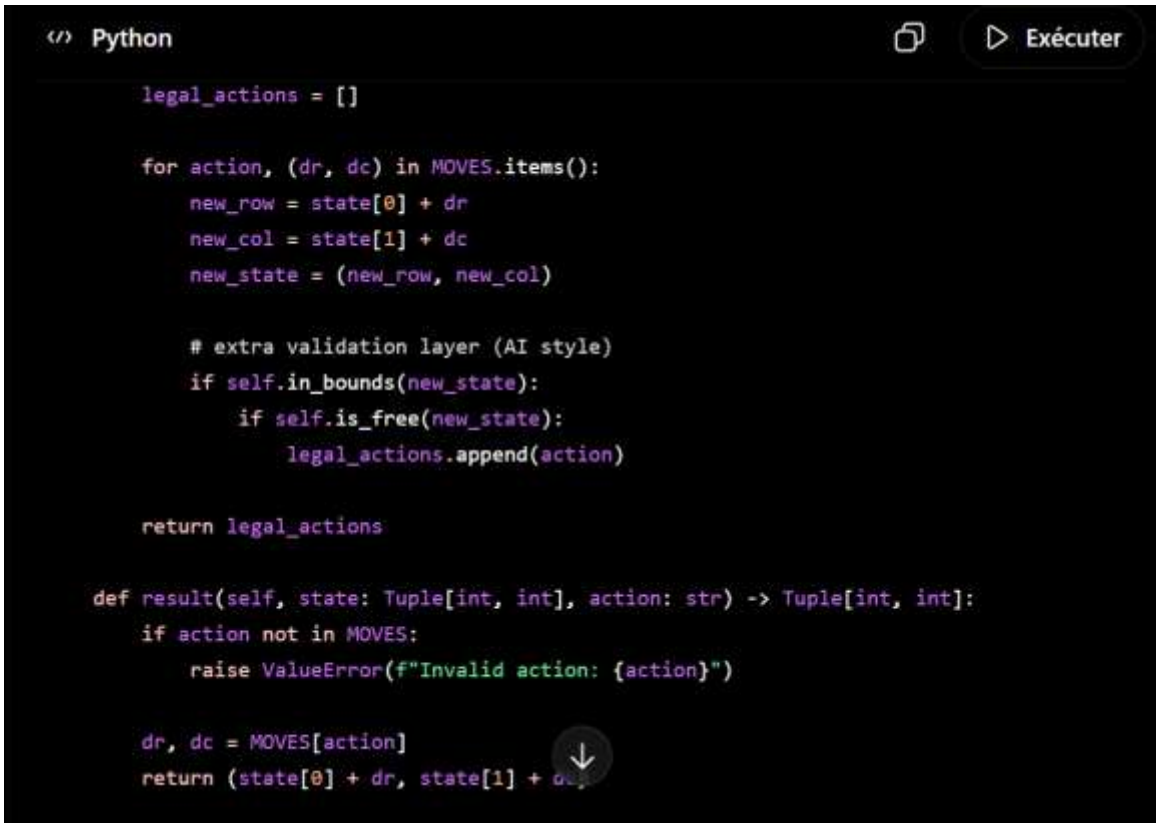
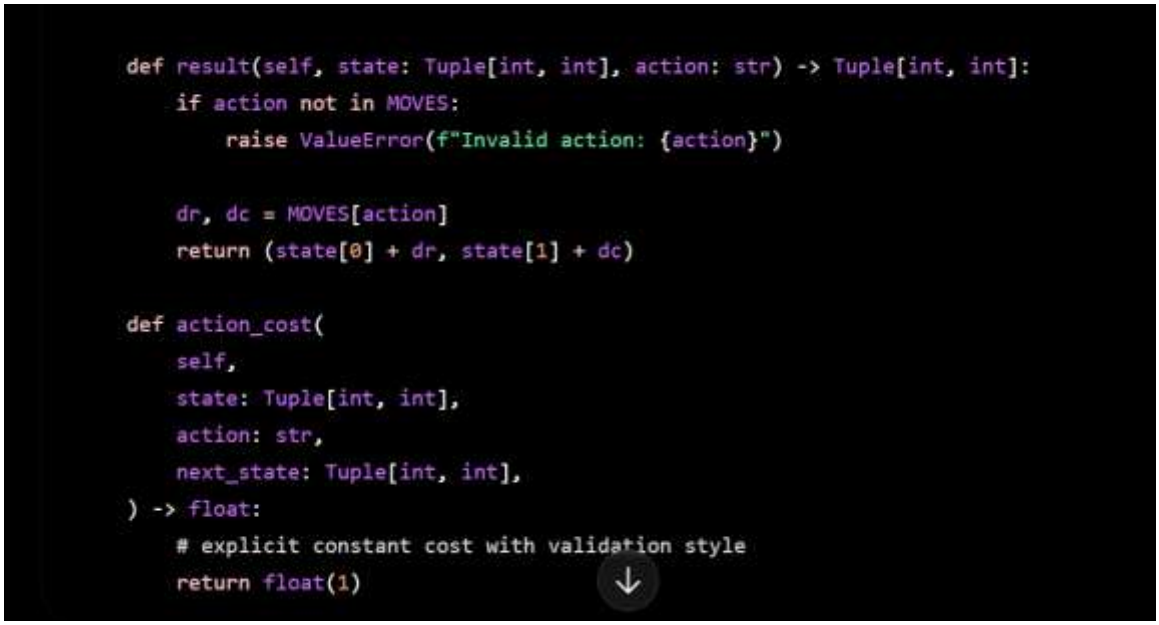
AI_Evidence:

Lab Part A:

AI suggested todo 1





```
    def result(self, state: Tuple[int, int], action: str) -> Tuple[int, int]:
        if action not in MOVES:
            raise ValueError(f"Invalid action: {action}")

        dr, dc = MOVES[action]
        return (state[0] + dr, state[1] + dc)

    def action_cost(
        self,
        state: Tuple[int, int],
        action: str,
        next_state: Tuple[int, int],
    ) -> float:
        # explicit constant cost with validation style
        return float(1)
```

```
MOVES = {
   "UP": (-1, 0),
   "DOWN": (1, 0),
   "LEFT": (0, -1),
   "RIGHT": (0, 1),
}


class GridProblem(Problem):
   def __init__(
      self,
      grid: List[List[int]],
      start: Tuple[int, int],
      goal: Tuple[int, int],
```

```
):
    self.grid = grid
    self.start = start
    self.goal = goal

    self.rows = len(grid)
    self.cols = len(grid[0])

def initial_state(self) -> Tuple[int, int]:
    return self.start

def is_goal(self, state: Tuple[int, int]) -> bool:
    # TODO 1
    return state == self.goal

def in_bounds(self, state: Tuple[int, int]) -> bool:
    row, col = state
    return 0 <= row < self.rows and 0 <= col < self.cols

def is_free(self, state: Tuple[int, int]) -> bool:
    row, col = state
    return self.grid[row][col] == 0

def actions(self, state: Tuple[int, int]) -> List[str]:
    # TODO 2
    legal_actions = []
```

```
    for action, (dr, dc) in MOVES.items():
        new_state = (state[0] + dr, state[1] + dc)

        if self.in_bounds(new_state) and self.is_free(new_state):
            legal_actions.append(action)

    return legal_actions

def result(self, state: Tuple[int, int], action: str) -> Tuple[int, int]:
    # TODO 3
    dr, dc = MOVES[action]
    return (state[0] + dr, state[1] + dc)

def action_cost(
    self,
    state: Tuple[int, int],
    action: str,
    next_state: Tuple[int, int],
) -> float:
    # TODO 4
    return 1
```

```
if problem.is_goal(node.state):
    return node, "success"

# depth limit check (AI explicit safety check)
if node.depth is None or node.depth >= limit:
    return None, "cutoff"

metrics["nodes_expanded"] += 1

cutoff_occurred = False

for child in self.expand(problem, node):

    # AI-style explicit cycle detection using set-like logic
    current = node
    visited_in_path = set()

    while current is not None:
        visited_in_path.add(current.state)
        current = current.parent
```

```
    if child.state in visited_in_path:
        continue

    # update stack size metric (AI redundant tracking)
    metrics["max_stack_size"] = max(
        metrics["max_stack_size"],
        current_stack_size + 1
    )

    try:
        result, status = self._recursive_dls(
            problem,
            child,
            limit,
            metrics,
            current_stack_size + 1
        )
    except RecursionError:
        return None, "cutoff"
```

Student revised code

```
# TODO 8


    if problem.is_goal(node.state):
        return node, "success"


    if node.depth >= limit:
        return None, "cutoff"


    metrics["nodes_expanded"] += 1


    cutoff_occurred = False


    for child in self.expand(problem, node):


        # skip cycles (path checking)
        temp = node
        cycle = False
        while temp is not None:
            if temp.state == child.state:
                cycle = True
                break
            temp = temp.parent


        if cycle:
            continue
```

```
    metrics["max_stack_size"] = max(metrics["max_stack_size"], current_stack_size + 1)

    result, status = self._recursive_dls(
        problem,
        child,
        limit,
        metrics,
        current_stack_size + 1
    )

    if status == "success":
        return result, "success"

    if status == "cutoff":
        cutoff_occurred = True

if cutoff_occurred:
    return None, "cutoff"
else:
    return None, "failure"
```

Reflection AI help

1. Meaning of "cutoff"

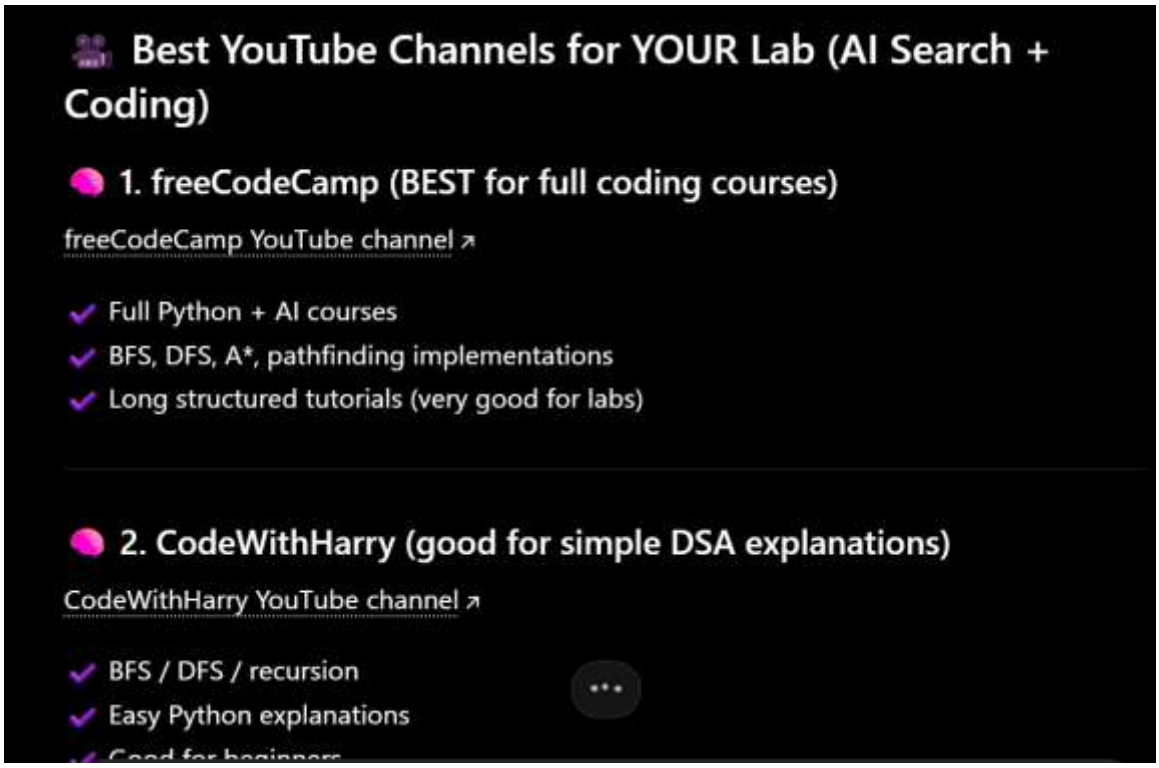
A **cutoff** means the search stopped because it reached the **depth limit** before finding the goal.
It does not mean the goal does not exist; it only means the algorithm could not go deeper in that branch due to the limit.

2. Why does DFS use a stack?

DFS uses a **stack** because it explores the most recently discovered node first (LIFO order). This allows DFS to go as deep as possible along one path before backtracking, which matches its depth-first behaviour.

4. Why does IDS repeat DLS with increasing limits?

Iterative Deepening Search (IDS) repeats Depth-Limited Search (DLS) with increasing depth limits to combine the benefits of both DFS and BFS.
It starts with a small depth limit and gradually increases it until the goal is found, ensuring completeness while avoiding excessive memory use.



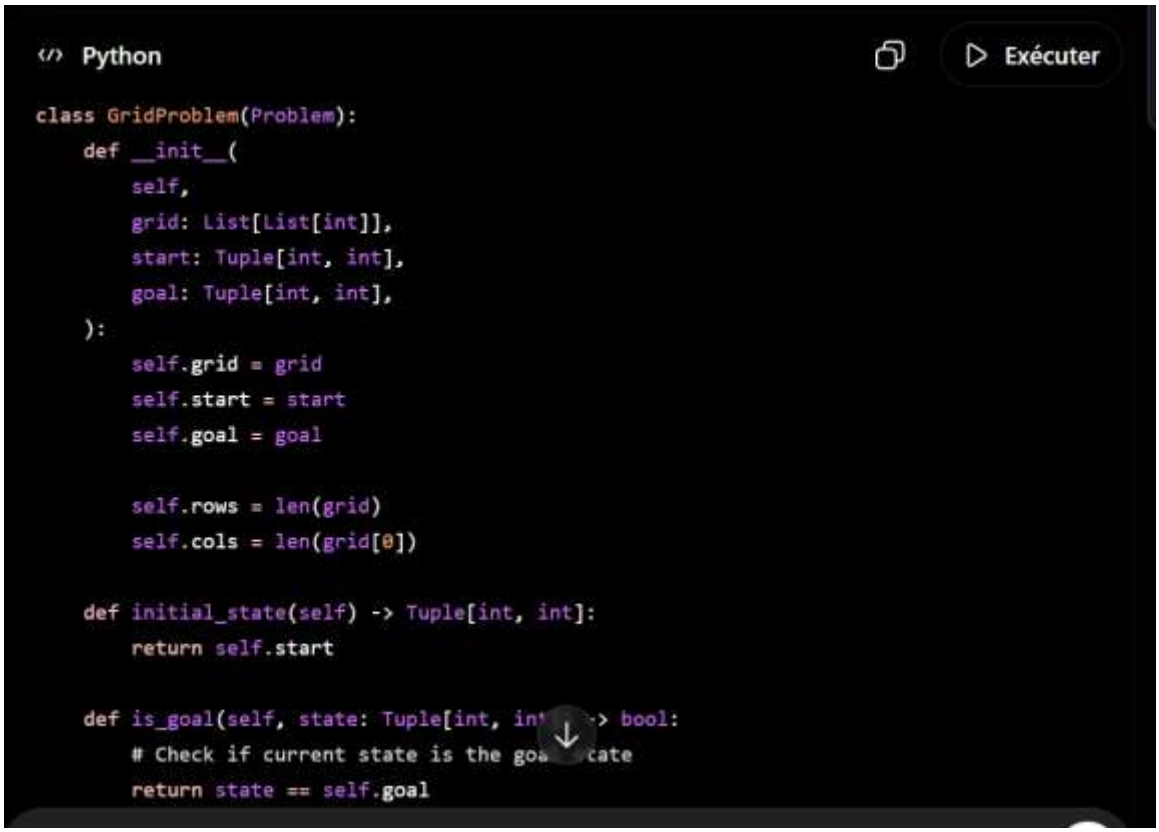
Lab Part B
Student first try

```
class GridProblem(Problem):

   def __init__(

      self,
```

```
    grid: List[List[int]],
    start: Tuple[int, int],
    goal: Tuple[int, int],
):
    self.grid = grid
    self.start = start
    self.goal = goal

    self.rows = len(grid)
    self.cols = len(grid[0])

def initial_state(self) -> Tuple[int, int]:
    return self.start

def is_goal(self, state: Tuple[int, int]) -> bool:
    # TODO 1
    return state == self.goal

def in_bounds(self, state: Tuple[int, int]) -> bool:
    row, col = state
    return row >= 0 and row < self.rows and col < self.cols

def is_free(self, state: Tuple[int, int]) -> bool:
    row, col = state
    return self.grid[row][col] == 0
```

```
    def actions(self, state: Tuple[int, int]) -> List[str]:
        # TODO 2
        legal_actions = []

        for action, (dr, dc) in MOVES.items():
            next_state = (state[0] + dr, state[1] + dc)

            if self.in_bounds(next_state) and self.is_free(next_state):
                legal_actions.append(action)

        return legal_actions

    def result(self, state: Tuple[int, int], action: str) -> Tuple[int, int]:
        # TODO 3
        dr, dc = MOVES[action]
        return (state[0] + dr, state[1] + dc)

    def action_cost(
        self,
        state: Tuple[int, int],
        action: str,
        next_state: Tuple[int, int],
    ) -> float:
        # TODO 4
        return 1
```

AI revision


Python
Exécuter
class GridProblem(Problem):
def __init__(
self,
grid: List[List[int]],
start: Tuple[int, int],
goal: Tuple[int, int],
):
self.grid = grid
self.start = start
self.goal = goal
self.rows = len(grid)
self.cols = len(grid[0])
def initial_state(self) -> Tuple[int, int]:
return self.start
def is_goal(self, state: Tuple[int, int]) -> bool:
# Check if current state is the goal state
return state == self.goal

Python

Exécuter

```python
        row, col = state
        return 0 <= row < self.rows and 0 <= col < self.cols

    def is_free(self, state: Tuple[int, int]) -> bool:
        row, col = state
        return self.grid[row][col] == 0

    def actions(self, state: Tuple[int, int]) -> List[str]:
        legal_actions = []

        for action, (dr, dc) in MOVES.items():
            next_state = (state[0] + dr, state[1] + dc)

            if self.in_bounds(next_state) and self.is_free(next_state):
                legal_actions.append(action)

        return legal_actions

    def result(self, state: Tuple[int, int], action: str) -> Tuple[int, int]:
        dr, dc = MOVES[action]
        return (state[0] + dr, state[1] + dc)
```

AI Refined part

Python Exécuter

```python
class BestFirstSearch(SearchAlgorithm):
    """Generic best-first search. Subclasses define the evaluation function f(n)."""

    algorithm_name = "BestFirst"

    def __init__(
        self,
        heuristic: Callable[[Tuple[int, int], Tuple[int, int]], float] = zero_heuristic,
    ):
        self.heuristic = heuristic

    def h(self, node: Node, problem: Problem) -> float:
        return self.heuristic(node.state, problem.goal)

    def evaluation(self, node: Node, problem: Problem) -> float:
        raise NotImplementedError("Subclasses must define evaluation(node, problem)")

    def search(self, problem: Problem) -> SearchResult:

        start = Node(problem.initial_state())
```

Python Exécuter

```python
sample_grid = [
    [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
    [1, 1, 1, 0, 1, 0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
    [0, 1, 1, 1, 1, 0, 1, 0, 1, 1],
    [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
    [0, 1, 1, 0, 1, 1, 1, 1, 1, 0],
    [0, 0, 1, 0, 0, 0, 0, 0, 1, 0],
    [1, 0, 1, 1, 1, 1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
    [0, 1, 1, 1, 1, 0, 0, 0, 1, 0],
]
```

📘 Informed Search Reflection (Explanation Version)

16.1 Heuristic Functions

The heuristic function h(n) estimates the cost from the current state to the goal state. It does not use real terrain information; instead, it is based on grid geometry using the coordinates of the current position and the goal.

Manhattan relaxation simplifies the problem by removing obstacles and assuming that every move has a uniform cost of 1. This turns the grid into an open space where movement is unrestricted.

16.2 Greedy Best-First Search

Greedy Best-First Search uses only the heuristic h(n) and ignores the path cost g(n). Because of this, it always chooses the state that appears closest to the goal, even if it leads through expensive or inefficient paths.

Although Greedy may expand fewer nodes than A*, this does not make it better. It can produce suboptimal solutions because it does not consider the cost already spent.